

TD Vue.js - Mini Todo App

Réponses complètes Q1 à Q14 + tableau comparatif final

Q1

Un fichier `.vue` sépare clairement le code en trois parties : ``script``, ``template`` et ``style``. C'est lisible et organisé. L'inconvénient est qu'il faut comprendre cette structure spéciale. En React `.tsx`, tout est dans JavaScript/TypeScript, donc c'est très flexible, mais le fichier peut vite devenir mélangé et moins lisible.

Q2

React ne propose pas de two-way binding natif parce qu'il préfère garder un flux de données explicite et contrôlé. La valeur vient du state, puis l'utilisateur modifie l'input avec ``onChange``. Cela rend le comportement plus clair, même si c'est plus long à écrire.

Q3

En React, il faut respecter l'immutabilité : on ne modifie pas directement le tableau, on crée une nouvelle version. En Vue, ``tasks.value.push(data)`` fonctionne parce que Vue utilise un système réactif basé sur des ``Proxy``. Le Proxy détecte la mutation du tableau et met automatiquement l'interface à jour.

Q4

``onMounted(fn)`` est souvent plus lisible car son nom indique directement le moment d'exécution. React utilise ``useEffect(fn, [])`` avec un tableau de dépendances pour gérer plusieurs cas : montage, mise à jour selon certaines variables, ou exécution à chaque rendu. C'est plus puissant, mais moins direct pour un débutant.

Q5

L'approche Vue avec les événements est plus proche du HTML natif, car on écoute un événement avec ``@delete``, comme on écouterait un clic avec ``@click``. React passe plutôt des fonctions en props, ce qui ressemble plus à une communication JavaScript directe.

Q6

En Vue, si on oublie d'écouter l'événement ``@delete``, l'application ne crash pas. L'enfant émet l'événement, mais comme aucun parent ne l'écoute, l'événement est simplement ignoré.

Q7

La logique n'est pas vraiment différente. React Router et Vue Router utilisent tous les deux des hooks/composables pour lire les paramètres et naviguer. Les noms changent surtout : ``useParams`` et ``useNavigate`` côté React, ``useRoute`` et ``useRouter`` côté Vue.

Q8

La séparation de Vue dans ``router/index.ts`` permet de centraliser les routes dans un seul fichier. C'est plus propre pour les projets moyens ou grands, car on retrouve rapidement toutes les pages et leurs chemins sans mélanger la configuration avec l'affichage.

Q9

Redux Toolkit demande plus de concepts : `createSlice`, `configureStore`, `Provider`, `useSelector`, `useDispatch`, `dispatch`, reducers/actions. Pinia est plus simple : `defineStore`, `useTaskStore`, state et actions directes. Donc Pinia demande moins de notions au départ.

Q10

`store.addTask(title)` est plus intuitif, car on appelle directement une méthode du store. Redux a quand même un avantage : son architecture est très stricte et ses DevTools sont très puissants, notamment avec le time-travel debugging.

Tableau de comparaison finale

Concept	React	Vue 3
State local	<code>const [x, setX] = useState(0)</code>	<code>const x = ref(0)</code>
Two-way binding	<code>value={x}` + `onChange={e => setX(e.target.value)}</code>	<code>v-model="x"</code>
Fetch au montage	<code>useEffect(() => { fetch() }, [])</code>	<code>onMounted(() => { fetch() })</code>
Ajouter au state	<code>setItems(prev => [...prev, data])</code>	<code>items.value.push(data)</code>
Boucle template	<code>items.map(i => <li key={i.id}>...)</code>	<code><li v-for="i in items" :key="i.id">...</code>
Conditionnel	<code>{condition && <div>...</div>}</code>	<code><div v-if="condition">...</div></code>
Event	<code>onClick={handler}</code>	<code>@click="handler"</code>
Props enfant	<code><Child name={x} /></code>	<code><Child :name="x" /></code>
Callback parent	<code><Child onDelete={handler} /></code>	<code><Child @delete="handler" /></code>
Router lien	<code><Link to='/x'>Lien</Link></code>	<code><RouterLink to='/x'>Lien</RouterLink></code>
Params URL	<code>const { id } = useParams()</code>	<code>const id = useRoute().params.id</code>
Navigate	<code>const nav = useNavigate(); nav('/')</code>	<code>const router = useRouter(); router.push('/')</code>
State global	<code>createSlice + useSelector + dispatch</code>	<code>defineStore + useTaskStore()</code>
Appeler action	<code>dispatch(addTask(title))</code>	<code>store.addTask(title)</code>
Style isolé	CSS Modules / `.module.css	<code><style scoped></code>

Q11

Les concepts presque identiques sont : composants, props, routes, paramètres d'URL, navigation, événements, state local, state global et fetch au montage. Les concepts fondamentalement différents sont : le two-way binding avec `v-model`, les mutations directes détectées par Proxy en Vue, l'immuabilité obligatoire en React, et la séparation `template/script/style` dans Vue contre JSX/TSX dans React.

Q12

Vue est plus « magique » parce que `v-model` et les mutations directes simplifient beaucoup le code. C'est un avantage pour apprendre vite et écrire moins de code. L'inconvénient est que certains mécanismes sont moins visibles, donc il faut comprendre la réactivité de Vue pour éviter les confusions.

Q13

Pour une app e-commerce avec 50+ pages, une équipe de 10 développeurs et un dashboard complexe, React peut être un bon choix grâce à son très grand écosystème, sa popularité en entreprise et ses nombreuses bibliothèques. Vue reste aussi très adapté, surtout si l'équipe veut une structure plus simple et plus lisible. Mon choix serait React si l'équipe cherche un maximum de bibliothèques et de profils disponibles, Vue si la priorité est la rapidité de développement et la simplicité.

Q14

Pour un débutant, je conseille Vue en premier. Sa syntaxe est plus proche du HTML, `v-model` est simple, les fichiers `.vue` sont bien organisés, et Pinia est plus facile à comprendre que Redux. React est très important à apprendre ensuite, mais il demande plus vite de comprendre JSX, hooks, immutabilité et gestion explicite du state.